
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

We introduce FLOWTIME, a conditional generative model for probabilistic forecasting of multivariate time series. Conditioned on a history of observed measurements and, optionally, on future covariates, we cast forecasting as the task of drawing samples from a learned conditional density over future trajectories. Concretely, we factorize the joint density of the future observations into an ordered product of one-step conditional densities, and parameterize each factor by a single shared flow that continuously transports a tractable base distribution to the predictive distribution of the next observation, conditioned on the encoded covariate context. We instantiate these transports within the flow matching (FM) framework, which affords scalable, simulation-free estimation of the underlying velocity field without integrating an ordinary differential equation at training time. Coupling this autoregressive factorization with the FM regression objective yields a model that inherits the properties of autoregressive predictors—accurate extrapolation beyond the training horizon, a compact parameterization, and well-calibrated predictive uncertainty—while retaining the capacity of modern transport-based generative models to represent complex, multi-modal conditional distributions. We evaluate FLOWTIME empirically on simulated stochastic dynamical systems and on real-world forecasting benchmarks.

1 INTRODUCTION

Probabilistic time series forecasting is a central problem in modern machine learning: given a record of past observations, the objective is to characterize the conditional distribution of future states of a dynamical system rather than a single point estimate. The problem arises across a broad set of scientific, industrial, and societal domains (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024).

A prominent line of work addresses this problem with deep generative models (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021), under which forecasting is formalized as sampling from a predictive density over future values conditioned on the observed past. In particular, diffusion and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020) have advanced the state of the art on several forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kolloviev et al., 2023; Meijer & Chen, 2024). Their predictive accuracy, however, is obtained at a substantial computational cost during both training and inference, owing to the many stochastic denoising steps required to draw a single sample.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) is a generative-modeling framework that generalizes and subsumes diffusion models while admitting a broader set of design choices. Whereas diffusion models generate samples through an iterative stochastic denoising chain applied along a discretized noising trajectory, FM specifies a deterministic probability path and learns the velocity field of a continuous normalizing flow that transports an arbitrary base distribution onto the target distribution. Because the probability path is prescribed rather than induced by a noising process, FM removes the need for hand-designed noise schedules and long ancestral sampling chains, which in turn yields more efficient training and sampling.

FM has recently been instantiated for time series modeling and probabilistic forecasting (Tamir et al., 2024; Hu et al., 2024; Kolloviev et al., 2024), reporting strong empirical accuracy together with improved computational efficiency relative to diffusion baselines. These approaches learn,

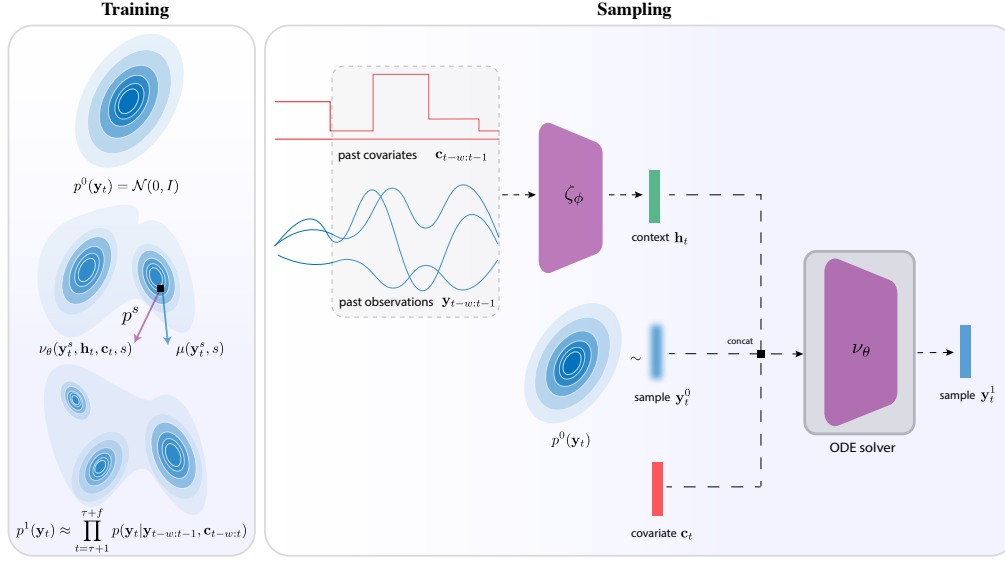


Figure 1: Overview of the training and sampling procedures of FLOWTIME. At training time, a probability path $(p^s)^{0 \leq s \leq 1}$ is prescribed to interpolate between a base distribution p^0 and the target distribution p^1 , and is generated by an associated velocity field μ . Training regresses a neural network ν_θ onto μ ; the network takes as input a sample y_t^s drawn from the probability path at flow step s , a context vector $\mathbf{h}_t$ that encodes past observations and covariates, and the current covariate $\mathbf{c}_t$. At inference time, a sample from the target distribution is obtained by drawing an initial state from p^0 and numerically integrating the learned velocity field with an ODE solver from $s = 0$ to $s = 1$.

in a single transport, the conditional distribution of an entire fixed-horizon future window given a fixed-horizon context window. This design admits fast, fully parallel training and sampling, but it exhibits three drawbacks: it extrapolates poorly beyond the horizon seen during training, it produces miscalibrated predictive uncertainty, and it poses a harder estimation problem, since the network must fit an intricate, jointly time- and flow-dependent velocity field over the full trajectory. The last of these difficulties is examined by Kollovich et al. (2024), who introduce informed Gaussian-process priors to regularize and accelerate optimization; that construction, however, is confined to the univariate setting.

We propose FLOWTIME, a direct alternative that applies FM to learn probabilistic forecasters which scale to high-dimensional multivariate data without recourse to informed priors. Rather than modeling the entire future window jointly, FLOWTIME factorizes the forecasting task into an ordered sequence of one-step conditional distributions and models the distribution of each future observation conditioned on past observations and covariates, using a single shared flow that transports a simple base distribution onto the predictive distribution of the next observation. This autoregressive construction lets FLOWTIME accommodate variable-length forecast horizons, extrapolate beyond the training horizon, and emit well-calibrated predictive uncertainty, while reducing the estimation problem to a family of comparatively simple one-step transports.

Empirically, FLOWTIME attains strong forecasting performance on both classical stochastic dynamical systems and real-world benchmarks, providing a practical and effective approach to probabilistic time series forecasting.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) is a generative-modeling framework that enables continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018) to be trained at previously unattainable scale. A CNF represents a data transformation as the solution of an ordinary differential

equation (ODE), yielding an invertible mapping whose likelihood is tractable via the instantaneous change-of-variables formula.

In common with CNFs, FM seeks a time-dependent diffeomorphism on the data space Ω , termed a flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. The flow maps a sample $\mathcal{X}^0 \sim p$ from a source distribution p to a target sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ distributed according to a prescribed target q , i.e., $\mathcal{X}^1 \sim q$. It is defined as the solution of the initial value problem

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

where $\mu: [0, 1] \times \Omega \rightarrow \Omega$ is the vector field that specifies the instantaneous velocity of the flow and generates a probability path $(p^s)_{0 \leq s \leq 1}$, each p^s being a distribution over Ω with boundary condition $p^0 = p$. The learning problem is therefore to identify a vector field μ whose induced path satisfies the terminal condition $p^1 = q$.

In contrast to CNFs, whose vector field is fit by likelihood maximization and thus requires solving and backpropagating through Eq. (1) at each training step, FM fits a parametric vector field ν_θ with parameters θ by regressing it against the analytically known velocity fields of a family of fixed conditional probability paths. This substitution yields a tractable objective, the conditional flow matching (CFM) loss, defined as

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0, 1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

Once this objective is minimized, samples from q are generated by integrating Eq. (1) with the learned field ν_θ . Because no gradients are propagated through the ODE solver, FM affords stable and scalable optimization for high-dimensional generative tasks such as image and video synthesis (Esser et al., 2024; Polyak et al., 2024). A range of probability-path constructions and conditioning schemes have been proposed, unifying several transport-based generative models within this framework (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023).

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ denote an n -dimensional observation at time index τ . Given a look-back window of l past measurements up to and including the current observation, $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, the task is to produce probabilistic forecasts of the f subsequent values $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$, optionally conditioned on a sequence of observed covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$ with $\mathbf{c}_\tau \in \mathbb{R}^c$. Formally, the objective is to learn a sampler for the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ from a dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$ of m instances.

We factorize this conditional distribution autoregressively over future time steps under a Markov assumption of order w , giving

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ denotes the history (window) size. The learning objective thus reduces to estimating the one-step conditional distribution appearing in Eq. (3). In place of the maximum-likelihood estimation conventionally used for autoregressive models, we estimate this conditional distribution within the flow matching framework (Lipman et al., 2022).

This autoregressive factorization confers four concrete advantages. First, it replaces the estimation of a single complex, high-dimensional joint distribution with the estimation of a family of lower-dimensional one-step conditionals $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$. Second, because ground-truth histories are available during training, each one-step conditional can be fit under teacher forcing, so all time steps are optimized in parallel. Third, the Markov order w acts as an explicit control that trades representational capacity against computational cost. Fourth, the factorization yields interpretable uncertainty quantification, as the chain of conditional distributions makes explicit how predictive uncertainty propagates forward in time.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For every future time step $t \in [\tau + 1, \tau + f]$, we prescribe a target probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$ indexed by the flow variable s , with boundary marginals $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and $p^1(\mathbf{y}_t) \approx$

$\prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$, where $p_{\mathcal{D}}$ denotes the empirical data distribution. This path is generated by a flow $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ that evolves along s according to the ordinary differential equation (ODE)

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the vector field generating the flow, chosen so that the induced marginals satisfy the continuity equation associated with $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$.

To obtain a tractable estimator, we express the marginal probability path as a mixture of conditional probability paths,

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t | \mathbf{z}) p(\mathbf{z}) d\mathbf{z} \quad (5)$$

where $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is a conditioning variable drawn from an arbitrary coupling $\pi_{0,1}$. Theorem 2 of Lipman et al. (2022) establishes that the marginal velocity field $\mu(\mathbf{y}_t, s)$ is recovered, in expectation over $\mathbf{z}$, from the conditional velocity fields $\mu(\mathbf{y}_t, s | \mathbf{z})$ associated with the conditional paths $p^s(\mathbf{y}_t | \mathbf{z})$; regressing onto the conditional fields therefore suffices to learn the marginal field.

Following Tong et al. (2023), we take $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ under the independent coupling $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are samples from p^0 and p^1 , respectively, and define the conditional probability path as the Gaussian bridge

$$p^s(\mathbf{y}_t | \mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

whose boundary marginals satisfy $p^0(\mathbf{y}_t | \mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t | \mathbf{z}) = p^1(\mathbf{y}_t)$ in the limit $\sigma^2 \rightarrow 0$. The corresponding conditional velocity field is the constant displacement $\mu(\mathbf{y}_t, s | \mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This construction yields probability paths that are straight lines between source and target in Euclidean space, together with closed-form conditional samples and velocities, which in turn makes both training and sampling efficient. Once the velocity field is learned, samples from the target conditional $p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ are obtained by integrating the ODE in Eq. (4) from $s = 0$ to $s = 1$.

3.2 TRAINING

As in the standard FM setting, we estimate the velocity field by regressing a neural network ν_θ with parameters θ onto the conditional field, using the CFM objective of Eq. 2. To condition each one-step transport on its history, we summarize the context window at time t into a fixed-dimensional context vector $\mathbf{h}_t \in \mathbb{R}^h$, computed as

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a sequence encoder with parameters ϕ . The resulting learning objective is

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t | \mathbf{z})} \|\mu(\mathbf{y}_t^s, s | \mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

which we minimize by drawing, from $\mathcal{D}$, an observation at a time index t together with its context window and covariates, and jointly optimizing ϕ and θ by stochastic gradient descent on $\mathcal{L}(\theta, \phi)$. This estimation scheme corresponds to teacher forcing (Williams & Zipser, 1989): the model is conditioned on ground-truth past observations rather than on its own predictions during training, which decouples the time steps, permits their optimization in parallel, and avoids the accumulation of prediction error at training time. The full procedure is summarized in Algorithm 1.

3.3 INFERENCE

At inference time, given a previously unseen history $\mathbf{Y}_l$ and its covariates $\mathbf{C}$, we generate a forecast over the f subsequent time steps. Sampling proceeds autoregressively via the rolling-window procedure of Algorithm 2: at each step the newly generated observation is appended to the conditioning window and used to form the context for the next step. Repeating the entire procedure yields an ensemble of trajectories, from which we estimate empirical predictive quantiles and hence the forecast uncertainty.

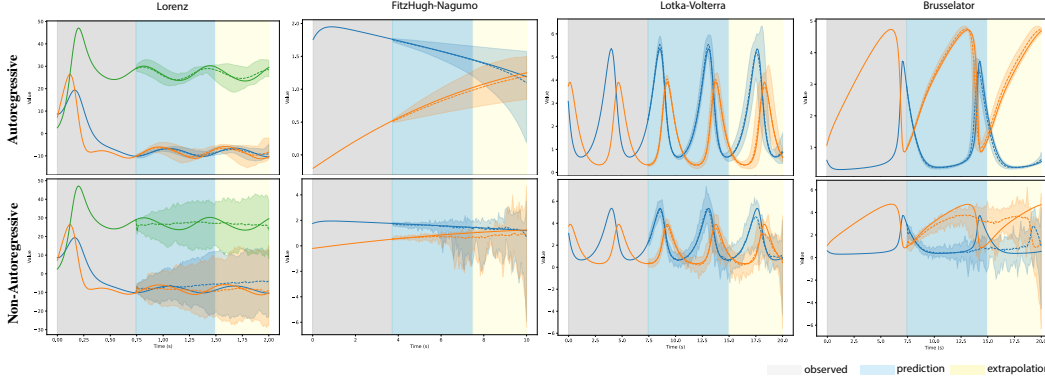


Figure 2: Forecast samples from autoregressive flow matching versus standard (non-autoregressive) flow matching on four dynamical systems. Solid lines denote the ground-truth trajectories; dashed lines denote the predictive mean, and the shaded band denotes the associated 95% confidence interval. Each panel is shown over both the prediction and the extrapolation regimes.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
- 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
- 5: Compute target: $\mu(\mathbf{y}_t^s, s | \mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
- 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s | \mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
- 7: Update θ, ϕ
- 8: **end while**
- 9: **return** ν_θ, ζ_ϕ

4 EXPERIMENTS

This section reports the empirical evaluation of FLOWTIME against a range of baselines, on both real-world datasets and synthetic data obtained by simulating stochastic dynamical systems.

4.1 DYNAMICAL SYSTEMS

Data We evaluated FLOWTIME on the forecasting of five stochastic dynamical systems: stochastic variants of the Lorenz system (Lorenz, 1963), the FitzHugh–Nagumo model (FitzHugh, 1961), the Lotka–Volterra system Lotka (1925), the Brusselator (Lefever & Prigogine, 1968), and the Van der Pol oscillator (van der Pol, 1926). For each system, trajectories were generated by numerically integrating the corresponding stochastic differential equations from randomly sampled initial condi-

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
- 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
- 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
- 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
- 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s), \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0, \quad s \in [0, 1]$
- 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
- 7: **end for**
- 8: **return** $\mathbf{Y}_f$

tions under independently drawn Brownian-motion realizations. Full specifications of the systems and of the data-generation protocol are given in Appendix A.1.

Implementation details For these experiments, the context encoder ζ_ϕ is a two-layer bidirectional LSTM with 64 hidden units, and the velocity network ν_θ is a multilayer perceptron with three hidden layers of width 64. The flow step s is mapped to a 16-dimensional Fourier positional embedding computed from a fixed bank of frequencies, and the Markov order w is set equal to the prediction length. Parameters were optimized by stochastic gradient descent with the Adam optimizer at a learning rate of 0.003 and a batch size of 128. Training to convergence required approximately 7 minutes on a single NVIDIA A100-SXM4-40GB GPU.

Baselines To isolate the contribution of the autoregressive factorization in FLOWTIME, we compared it against a non-autoregressive flow-matching baseline that fits the joint distribution of the full future window directly, conditioned on past observations and covariates, i.e. $p^1 = p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$. This baseline constitutes a controlled ablation in which the only varied factor is the presence or absence of the autoregressive decomposition, and it closely mirrors the formulation of Kolloviev et al. (2024). A complete description is deferred to Appendix B.

Evaluation To quantify extrapolation beyond the training horizon, we partitioned each trajectory into three consecutive, non-overlapping segments: (1) an observation window supplied to the model as history; (2) a prediction window used as the target during training; and (3) an extrapolation window withheld entirely at training time. Probabilistic calibration was measured by the continuous ranked probability score (CRPS) (Winkler et al., 1996), and point-forecast accuracy by the normalized root-mean-square error (NRMSE) (Hyndman & Koehler, 2006); for both metrics lower values are better.

Table 1: Comparison of the autoregressive (FLOWTIME) and non-autoregressive (Non-AR) factorizations of our method on five stochastic dynamical systems, reported as CRPS and NRMSE (lower is better) over the test set under both the prediction and extrapolation regimes. Each entry is the mean $\pm$ standard deviation across five random seeds; the best value in each column-block is shown in **bold**.

System	Factorization	Prediction		Extrapolation	
		CRPS $\downarrow$	NRMSE $\downarrow$	CRPS $\downarrow$	NRMSE $\downarrow$
Lorenz	Non-AR	0.147 $\pm$ 0.012	0.242 $\pm$ 0.018	0.120 $\pm$ 0.009	0.301 $\pm$ 0.024
	FLOWTIME	0.090 $\pm$ 0.008	0.017 $\pm$ 0.003	0.137 $\pm$ 0.011	0.048 $\pm$ 0.006
FitzHugh-Nagumo	Non-AR	0.165 $\pm$ 0.014	0.131 $\pm$ 0.011	0.214 $\pm$ 0.016	0.306 $\pm$ 0.022
	FLOWTIME	0.144 $\pm$ 0.013	0.090 $\pm$ 0.007	0.146 $\pm$ 0.012	0.278 $\pm$ 0.019
Lotka-Volterra	Non-AR	0.130 $\pm$ 0.011	0.266 $\pm$ 0.019	0.136 $\pm$ 0.012	0.502 $\pm$ 0.033
	FLOWTIME	0.126 $\pm$ 0.010	0.189 $\pm$ 0.015	0.123 $\pm$ 0.009	0.339 $\pm$ 0.026
Brusselator	Non-AR	0.217 $\pm$ 0.016	0.658 $\pm$ 0.042	0.238 $\pm$ 0.019	1.046 $\pm$ 0.057
	FLOWTIME	0.125 $\pm$ 0.011	0.031 $\pm$ 0.004	0.109 $\pm$ 0.008	0.023 $\pm$ 0.003
Van der Pol	Non-AR	0.143 $\pm$ 0.012	0.161 $\pm$ 0.014	0.211 $\pm$ 0.018	0.510 $\pm$ 0.031
	FLOWTIME	0.225 $\pm$ 0.017	0.051 $\pm$ 0.005	0.282 $\pm$ 0.021	0.101 $\pm$ 0.009

Results Table 1 reports the comparison. The autoregressive factorization in FLOWTIME yields a consistent improvement in forecasting performance across the five stochastic systems. On the prediction task, the AR variant attains lower NRMSE than the non-AR baseline on all five systems; the largest reductions occur on Lorenz (0.017 vs. 0.242) and Brusselator (0.031 vs. 0.658), corresponding to relative error reductions of 93% and 95%, respectively. The AR variant also achieves lower CRPS on four of the five systems, the sole exception being the Van der Pol oscillator, on which the non-AR baseline attains better probabilistic calibration. Critically, the advantage carries over to the extrapolation regime: the AR factorization retains lower NRMSE on every system, indicating stronger generalization beyond the training horizon. The effect is most pronounced on the

Brusselator, where the extrapolation NRMSE is reduced by 98% (0.023 vs. 1.046). Taken together, these results indicate that explicitly modeling the temporal dependence through the autoregressive factorization enables FLOWTIME to more faithfully capture the underlying dynamics of complex stochastic systems, improving both point accuracy and uncertainty calibration, and most markedly under extrapolation.

4.2 REAL-WORLD DATASETS

Data We evaluated FLOWTIME on the forecasting of several univariate and multivariate time series sampled at different frequencies (hourly and daily), drawn from GluonTS (Alexandrov et al., 2020). Specifically, we used the Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019) datasets. Their properties are detailed in Appendix A.2.

Implementation details Here the context encoder ζ_ϕ is a three-layer bidirectional LSTM with 64 hidden units. The velocity network ν_θ is a residual network of five residual blocks, each built from one-dimensional convolutional layers with gated activation functions (Van Den Oord et al., 2016; Kong et al., 2020); the architecture follows that of Rasul et al. (2021) but uses fewer residual blocks. The flow step s is encoded as a 32-dimensional Fourier positional embedding over a fixed frequency bank, and the same embedding scheme is applied to the time-dependent covariates c . Parameters were optimized by stochastic gradient descent with the Adam optimizer at a learning rate of 0.001 and a batch size of 128. Depending on the dataset, training to convergence required between 40 and 100 minutes on a single NVIDIA A100-SXM4-40GB GPU.

Baselines We compared FLOWTIME against a set of established baselines spanning three families. The classical statistical methods are Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008). The deterministic and autoregressive deep-learning methods are DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016). The generative baselines are the diffusion-based models TSDiff (Kollovich et al., 2023), SSSD (Alcaraz & Strodthoff, 2022), and TimeGrad (Rasul et al., 2021), together with the flow-matching model TSFlow (Kollovich et al., 2024).

Table 2: Test-set CRPS (lower is better) for the competing models on five real-world datasets. For FLOWTIME, entries report the mean $\pm$ standard error over five independent retraining-and-evaluation runs; baseline entries are reproduced from Kollovich et al. (2024). The best score in each column is shown in **bold** and the second best is underlined.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060 $\pm$ 0.000	0.013 $\pm$ 0.000	0.512 $\pm$ 0.000	0.221 $\pm$ 0.000	0.423 $\pm$ 0.000
ARIMA	0.344 $\pm$ 0.000	0.013 $\pm$ 0.000	0.558 $\pm$ 0.003	0.486 $\pm$ 0.000	0.654 $\pm$ 0.000
ETS	0.056 $\pm$ 0.000	0.008 $\pm$ 0.000	0.550 $\pm$ 0.000	0.492 $\pm$ 0.000	0.651 $\pm$ 0.000
DeepAR	0.051 $\pm$ 0.000	0.013 $\pm$ 0.004	0.429 $\pm$ 0.015	0.103 $\pm$ 0.000	0.215 $\pm$ 0.003
WaveNet	0.058 $\pm$ 0.008	0.012 $\pm$ 0.001	0.360 $\pm$ 0.000	0.099 $\pm$ 0.000	0.207 $\pm$ 0.000
CSDI	0.051 $\pm$ 0.000	0.013 $\pm$ 0.000	0.360 $\pm$ 0.000	0.152 $\pm$ 0.000	0.318 $\pm$ 0.012
SSSD	0.048 $\pm$ 0.000	<u>0.010</u> $\pm$ 0.000	0.354 $\pm$ 0.024	0.107 $\pm$ 0.002	<u>0.209</u> $\pm$ 0.000
TSFlow	<u>0.045</u> $\pm$ 0.001	0.009 $\pm$ 0.001	<u>0.343</u> $\pm$ 0.002	0.083 $\pm$ 0.000	0.227 $\pm$ 0.000
FLOWTIME	0.042 $\pm$ 0.001	0.009 $\pm$ 0.001	0.284 $\pm$ 0.002	<u>0.089</u> $\pm$ 0.000	0.243 $\pm$ 0.002

Results Table 2 reports the CRPS of FLOWTIME and the baselines across the five real-world datasets. FLOWTIME attains the lowest CRPS on three of the five datasets: Electricity, Exchange, and Solar. On Electricity it achieves a CRPS of 0.042, a 6.7% relative reduction over the second-best model, TSFlow (0.045). On Exchange it matches the best baseline, TSFlow, at a CRPS of 0.009, while improving substantially over the classical statistical methods and the remaining deep-learning models. The largest margin is observed on Solar, where FLOWTIME reaches a CRPS of 0.284, a 17.2% relative reduction over the next-best model, TSFlow (0.343). On Traffic, FLOWTIME ranks second with a CRPS of 0.089, marginally behind TSFlow (0.083) but ahead of all remaining statistical and deep-learning baselines. On Wikipedia it remains competitive at a CRPS of 0.243, trailing only WaveNet (0.207) and SSSD (0.209). Overall, the autoregressive flow-matching formulation in

FLOWTIME delivers state-of-the-art or competitive CRPS across a heterogeneous set of real-world forecasting tasks.

5 DISCUSSION

We introduced FLOWTIME, a probabilistic forecasting model that couples an autoregressive factorization with flow matching to produce accurate, well-calibrated predictions for multivariate time series. By decomposing the forecasting task into an ordered sequence of one-step conditional distributions, FLOWTIME reconciles the temporal dependence structure of sequential data with the distributional expressiveness of flow-based generative models. Across simulated stochastic dynamical systems and real-world datasets, FLOWTIME attains performance competitive with state-of-the-art baselines on demanding forecasting benchmarks. Relative to non-autoregressive flow-matching models that transport the entire future window jointly, the autoregressive decomposition yields measurable gains in uncertainty calibration and predictive-distribution coverage, and it generalizes robustly to out-of-distribution conditions—a property of practical importance when deployment-time inputs depart from the training distribution.

Several limitations remain. First, autoregressive sampling is inherently sequential and therefore incurs greater inference-time cost than fully parallel non-autoregressive models, which may constrain use in latency-sensitive settings. Second, the present implementation performs flow matching directly in the observation space, which may be suboptimal for high-dimensional or partially observed data. These observations suggest several directions for future work: formulating the transport in a learned latent space, extending the construction to irregularly sampled series, and applying the method to broader problems in scientific modeling and decision-making under uncertainty. In sum, FLOWTIME advances probabilistic time series forecasting and illustrates the utility of flow-based generative models for jointly improving point accuracy and uncertainty quantification.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
- Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>, 7(1):62, 2017.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.
- Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.

432 Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas,
433 Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function
434 RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–
435 1910. PMLR, 2019.

436 Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD:
437 Free-Form continuous dynamics for scalable reversible generative models. In *International Con-
438 ference on Learning Representations*, 2018.

439 Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in
440 Neural Information Processing Systems*, 33:6840–6851, 2020.

441 Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen.
442 Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL
443 <http://arxiv.org/abs/2411.07506>.

444 Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential
445 Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.

446 Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International
447 Journal of Forecasting*, 22(4):679–688, 2006.

448 Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep varia-
449 tional Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint
450 arXiv:1605.06432*, 2016.

451 Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang,
452 and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for proba-
453 bilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–
454 28364, 2023.

455 Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow
456 matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth
457 International Conference on Learning Representations*, 2024.

458 Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile
459 diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.

460 Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term
461 temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference
462 on Research & Development in Information Retrieval*, pp. 95–104, 2018.

463 R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal
464 of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.

465 Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical
466 Transactions of the Royal Society A*, 379(2194):20200209, 2021.

467 Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow match-
468 ing for generative modeling. In *The Eleventh International Conference on Learning Representa-
469 tions*, 2022.

470 Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data
471 with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.

472 Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.

473 Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.

474 Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv
475 preprint arXiv:2401.03006*, 2024.

476 Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv
477 Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media founda-
478 tion models. *arXiv e-prints*, pp. arXiv–2410, 2024.

-
- 486 Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Mul-
487 tivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint*
488 *arXiv:2002.06103*, 2020.
- 489 Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising dif-
490 fusion models for multivariate probabilistic time series forecasting. In *International Conference*
491 *on Machine Learning*, pp. 8857–8868. PMLR, 2021.
- 492 David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus.
493 High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances*
494 *in Neural Information Processing Systems*, 32, 2019.
- 495 David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic
496 forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):
497 1181–1191, 2020.
- 498 Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised
499 learning using nonequilibrium thermodynamics. In *International Conference on Machine Learn-*
500 *ing*, pp. 2256–2265. PMLR, 2015.
- 501 Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben
502 Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint*
503 *arXiv:2011.13456*, 2020.
- 504 Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow match-
505 ing for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Gener-*
506 *ative Modeling*, 2024.
- 507 Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based
508 diffusion models for probabilistic time series imputation. *Advances in Neural Information Pro-*
509 *cessing Systems*, 34:24804–24816, 2021.
- 510 Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrid Rector-
511 Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models
512 with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.
- 513 Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves,
514 Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for
515 raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.
- 516 B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical*
517 *Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.
- 518 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
519 neural networks. *Neural Computation*, 1(2):270–280, 1989.
- 520 Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B
521 Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring
522 rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.
- 523 Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time
524 series foundation models: Generalizing time series representation with large language model. In
525 *ArXiv:2104.00164*, 2024.

A DATASETS

A.1 DYNAMICAL SYSTEMS

We generated training and test data for five classical stochastic dynamical systems—Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator, and Van der Pol—by numerically integrating their governing stochastic differential equations (SDEs). Each system admits the general Itô form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \boldsymbol{\Theta}) dt + \boldsymbol{\sigma} d\mathbf{W}(t),$$

where f is the deterministic drift, $\boldsymbol{\Theta}$ the vector of system parameters, $\boldsymbol{\sigma}$ a constant diffusion coefficient, and $\mathbf{W}(t)$ a standard Brownian motion. For each system, we drew 2400 initial conditions from a uniform distribution over the range specified in Table 3 and integrated the SDE from each initial condition using the Euler–Heun scheme with a fixed step size. Each resulting trajectory was recorded at 200 equally spaced time points, partitioned into the first 75 points as observed history, the next 75 as the prediction window, and the final 50 as the extrapolation window. The 2400 trajectories were split into 2000 for training and 400 for testing.

Table 3: Specifications of the stochastic dynamical systems. $\mathbf{f}(\mathbf{x})$ is the drift function, $\boldsymbol{\Theta}$ the system parameters, $\boldsymbol{\sigma}$ the constant diffusion coefficient, and $p(\mathbf{x}(0))$ the distribution of initial conditions. Each system was integrated over the interval $[t_0, t_1]$ at 200 steps using the Euler–Heun scheme.

System	$f(\mathbf{x})$	$\boldsymbol{\Theta}$	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh–Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka–Volterra	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\beta = 0.9$			
		$\gamma = 0.8$			
Brusselator	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$A = 1.0$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_2 = B x_1 - x_1^2 x_2$	$B = 3.0$			
		$\delta = 1.8$			
Van der Pol	$f_1 = x_2$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$				

A.2 REAL-WORLD DATASETS

We evaluated our model on five datasets from GluonTS (Alexandrov et al., 2020), using the preprocessed versions of Salinas et al. (2019). Their properties are summarized in Table 4.

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting We retain the forecasting setup of Section 3: given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$, the goal is to model the conditional distribution $p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$. In contrast to the

Table 4: Properties of the datasets used in the experiments: observation dimension n , domain Ω , sampling frequency, number of training time steps, and prediction length.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

autoregressive factorization, however, this baseline models the joint distribution of the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$ directly, without any temporal decomposition:

$$p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f} | \mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation preserves the cross-temporal correlations among all future time steps, at the cost of having to estimate a single high-dimensional distribution.

Training We construct a probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ that transports the prior $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ onto the target $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$, where Σ is a block-diagonal covariance matrix. The prior admits an interpretation as a collection of per-dimension independent Brownian motions, each a stochastic process $W: [\tau + 1, \tau + f] \rightarrow \mathbb{R}^n$.

The conditional probability path is defined as a linear interpolation bridge perturbed by Brownian noise:

$$p^s(\mathbf{Y}_f | \mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$ with $\mathbf{Y}_f^0 \sim p^0$ and $\mathbf{Y}_f^1 \sim p^1$. The associated conditional velocity field is

$$\mu(\mathbf{Y}_f, s | \mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2} \Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

In the limit $\sigma^2 \rightarrow 0$, this collapses to the straight-path velocity $\mu(\mathbf{Y}_f, s | \mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. We fit a neural velocity field ν_θ that acts on the entire future trajectory. The context encoding parallels Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ now ingests both the past and the future covariates. The training objective is

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f | \mathbf{z})} \left\| \mu(\mathbf{Y}_f^s, s | \mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s) \right\|^2 \quad (13)$$

where ν_θ is realized as a sequence model (e.g., a Transformer or an RNN) applied over the whole trajectory. Sampling requires integrating the trajectory-level ODE

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

with a numerical solver. This yields joint samples from $p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$ without any autoregressive decomposition.

Implementation Details We applied this baseline to the dynamical-systems forecasting task. Here the velocity network ν_θ is a four-layer bidirectional LSTM with 128 hidden units, and all training hyperparameters are those of Section 4.1. Training to convergence required approximately 2 minutes on a single NVIDIA A100-SXM4-40GB GPU.

Key Differences from Autoregressive Variant The non-autoregressive baseline differs from the autoregressive formulation of FLOWTIME along three axes. First, in the factorization of the target: the autoregressive method decomposes the joint distribution into a Markov chain of one-step conditionals, whereas the non-autoregressive baseline treats the full future trajectory $\mathbf{Y}_f$ as a single high-dimensional random variable, preserving cross-temporal dependencies but requiring estimation of a more complex distribution over $\mathbb{R}^{n \times f}$. Second, in computational profile: the non-autoregressive variant passes entire trajectories through a sequence model to evaluate the velocity field, enabling all future time points to be generated in parallel but incurring higher memory cost. Third, in the treatment of uncertainty: the autoregressive approach models error accumulation explicitly through the chain of conditional distributions, whereas the non-autoregressive baseline captures joint uncertainty via trajectory-level sampling but has no explicit mechanism for error-accumulation dynamics. The two designs thus present complementary trade-offs—the autoregressive variant offers interpretable uncertainty quantification and efficient window-based computation under a bounded-order Markov assumption, while the non-autoregressive baseline retains the full temporal correlation structure at higher computational cost and with less explicit uncertainty dynamics.

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
- 3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$
- 5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
- 6: Compute target $\mu(\mathbf{Y}_f^s, s|\mathbf{z})$
- 7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
- 8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ

- 1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
- 2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
- 3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
- 4: Return $\psi(\mathbf{Y}_f, 1)$

C EVALUATION METRICS

This section specifies the metrics used to assess forecast quality. Given an n -dimensional target series $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ together with the predictive distribution—or samples drawn from it—produced by the model, we report two complementary metrics: the Continuous Ranked Probability Score (CRPS), which assesses probabilistic calibration, and the Normalized Root-Mean-Square Error (NRMSE), which assesses point-forecast accuracy.

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

The CRPS is a proper scoring rule widely used to assess probabilistic forecasts. For a univariate random variable X with cumulative distribution function (CDF) F_X and an observed realization x , it is defined as

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ denotes the indicator function. The CRPS thus measures the squared discrepancy between the predictive CDF and the degenerate CDF at the observation, integrated over the support; lower values correspond to predictive distributions that are jointly better calibrated and sharper.

For time series forecasting, we evaluate the CRPS at each step of the forecast horizon and average over the horizon,

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ is the predicted CDF—or its empirical estimate from samples—at horizon step k , and $y_{\tau+k}$ is the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

The Root-Mean-Square Error (RMSE) is a standard measure of point-forecast accuracy. For predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ and corresponding ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, it is defined as

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

To render the RMSE scale-invariant and thus comparable across datasets and series of differing magnitude, we adopt the normalized RMSE (NRMSE), obtained by dividing the RMSE by the

standard deviation of the ground-truth series, σ_y :

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observed series. An alternative normalization, appropriate in some applications, divides instead by the data range ($\max(y) - \min(y)$). A lower NRMSE denotes higher point-forecast accuracy relative to the intrinsic variability of the series.